哈爾濱工程大學

第 09 讲 / 全 17 讲

C 语言二维数组与字符数组详解

一张表装下全班成绩，一队字符拼成一个名字

二维数组

行与列两个下标，一张表就是它

双重循环

外层走行，内层走列，扫完整张表

字符数组

字符排成队，末尾要有结束标志

典型应用

求和、求平均、找最大、转置

主讲教师：郎大鹏

本节目标

哈爾濱工程大學

1. 说清二维数组「行 + 列」的含义，会写 int a[3][4]
2. 会用分行赋初值、连续赋值、部分初始化三种写法
3. 会用双重 for 循环把一张表读进来、再整齐打印出去
4. 会按行求总分、按列求平均，算得出全班总平均
5. 会用 max 、 row 、 col 三个变量找出最大元素和它的位置
6. 懂字符数组与字符串的关系，记住末尾的 '\0' 结束标志

01

二维数组基础

定义 · 内存排布 · 初始化 · 读入 · 输出

把一维数组的一排格子，扩展成一张有行有列的表

哈爾濱工程大學

一维数组不够用了

本课第一张表

哈尔滨工程大学

一个班 5 个人，每人 3 门课 —— 一维数组只是一条线，装不下这张表。

姓名	高数	英语	物理
张三	72	85	91
李四	66	90	55
王五	**78**	**73**	**82**
赵六	88	61	79
孙七	95	80	87

15 个成绩 —— 起 15 个变量名？太傻了。

一维数组

一条线，只装得下一个人的 3 门课

二维数组

一张表，5 行 3 列，全班一次装下

行 · 列

高亮那行是王五： s[2][1] 就是他的英语 73

二维数组的定义

前面是行，后面是列

哈尔滨工程大学

```
// 3 行 4 列，一共 12 个 int 格子
int a[3][4];
// 5 个学生，每人 3 门课
int s[5][3];
// 用的时候也是两个下标
a[0][0] = 72;  // 第 0 行第 0 列
a[2][3] = 69;  // 最后一行最后一列
s[2][1] = 73;  // 王五的英语
// 下标统统从 0 开始数
// a 的行下标：0 1 2
// a 的列下标：0 1 2 3
// 元素总个数 = 3 * 4 = 12
```

定义时只看行数和列数；用的时候先写行、再写列 —— 顺序反了意思就变了。

两个方括号

int a[3][4] ：第一个 3 是行数，第二个 4 是列数。

a[行][列]

访问时也是先写行、再写列，顺序不能颠倒。

合法范围

行 0 ~ 行数 -1 ，列 0 ~ 列数 -1 ； a[3][4] 最大到 a[2][3] 。

内存里是怎么放的

按行存放，一格挨一格

二维只是「写法」：内存里它仍是一长排，先放完第 0 行，再放第 1 行。

a[0][0] 第 1 个	**a[0][1]** 第 2 个	**a[0][2]** 第 3 个	**a[0][3]** 第 4 个
a[1][0] 第 5 个	**a[1][1]** 第 6 个	**a[1][2]** 第 7 个	**a[1][3]** 第 8 个
a[2][0] 第 9 个	**a[2][1]** 第 10 个	**a[2][2]** 第 11 个	**a[2][3]** 第 12 个

顺序：一行的 4 个放完，再放下 4 个

所以「按行遍历」正好顺着内存走一遍，一格不跳。

先横后竖

a[0][3] 后面紧跟着 a[1][0]，不是 a[3][0]。

a[i][j] 在哪

前面有 i 整行再加 j 个，即 i * 4 + j 个。

为什么关心

两个下标只是一长排格子的另一种数法，格子还是那 12 个。

二维数组的初始化

三种写法，一种推荐

哈尔滨工程大学

```
// 写法一：一层大括号写一行，推荐
int a[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
// 写法二：一长串，按顺序往里填
int b[2][3] = {1, 2, 3, 4, 5, 6};
// 写法三：只给一部分，其余补 0
int c[2][3] = {{1, 2}};
// c 实际是：1 2 0 然后 0 0 0
int d[2][3] = {0};   // 全部清零
```

三种写法效果完全一样。日常优先写法一：行和列的对应关系一眼看得清。

分行赋初值

一层大括号正好是一行，行数、列数都摆在那儿。

连续赋值

按内存顺序填，每填满 3 个就自动折到下一行。

部分初始化

没写到的位置自动补 0，不写就是 0，不是随机值。

逐元素赋值与循环读入

```c
// 一个个写，格子一多就写不完
int a[2][3];
a[0][0] = 1;
a[0][1] = 2;
// 用双重循环读入整张表
int i, j;
for (i = 0; i < 2; i++) {
    for (j = 0; j < 3; j++) {
        scanf("%d", &a[i][j]);
    }
}
```

读入顺序和内存顺序一致：先把第 0 行 3 个数读完，再读第 1 行 3 个数。

外层管行

i 从 0 走到 1 ，一次锁定一行，这一行不读完不走。

内层管列

j 从 0 走到 2 ，把这一行的三个格子挨个填上。

别忘了 &

scanf 要写 &a[i][j] ；漏了 & 程序会直接崩。

输出成一张整齐的表

换行时机是关键

```
//  一行一行打印，列要对齐
for (i = 0; i < 2; i++) {
    for (j = 0; j < 3; j++) {
        printf("%4d", a[i][j]);
    }
    //  这一行走完：输出一个换行
}
//  换行怎么表示：一个转义字符
//  即反斜杠加字母 n，写在格式串里
//  写在内层里：每个数都另起一行
//  写在外层外：整张表挤成一行
//  正确位置：内层之后、外层之内
```

%4d 干什么

每个数固定占 4 格宽，一位数和三位数也能对齐。

换行放哪

内层循环结束、外层循环内部 —— 一行打印完才换。

打印效果

```
   1   2   3
   4   5   6
```

02

二维数组的应用

求和 · 求平均 · 找最大 · 转置 · 相加

同一张表，换个循环顺序就换了一种统计口径

哈爾濱工程大學

遍历求全班总平均

先全加一遍，再除以个数

```
int s[3][4] = {   // 3 人 4 门课
    {72, 85, 91, 64},
    {66, 90, 55, 88},
    {78, 73, 82, 69}
};
int i, j, sum = 0;
for (i = 0; i < 3; i++) {
    for (j = 0; j < 4; j++) {
        sum = sum + s[i][j];
    }
}
// 12 个成绩加起来是 913
```

平均分写 sum / 12.0 得 76.08 ；写 sum / 12 只能得到整数 76 。

累加器先清零

sum 一定要初始化成 0 ，否则从垃圾值开始加。

圈数要数清

外层 3 圈、内层 4 圈，正好把 12 个格子走一遍。

个数从哪来

总个数 = 行数 × 列数 = 3 × 4 = 12 。

按行求和：每人总分

一行的四个数归一个人

哈尔滨工程大学

```
//  一行算一个人：固定行，走遍列
int i, j;
for (i = 0; i < 3; i++) {
    int sum = 0;    //  换人！清零
    for (j = 0; j < 4; j++) {
        sum = sum + s[i][j];
    }
    printf("%d", sum);
}
```

sum 写在第一层循环外面，三个人的分就加成一坨了。

位置决定口径

累加器放在外层循环体内，就是「一行一个值」。

算出来的总分

张三	312
李四	299
王五	302

打印别忘换行

三个总分要一行一个打印。

按列求和：每门课平均

一列的三个数归一门课

```
int i, j;
int col[4] = {0};  // 4 门课总分
for (i = 0; i < 3; i++) {
    for (j = 0; j < 4; j++) {
        col[j] = col[j] + s[i][j];
    }
}
// 总分: 216 248 228 221
for (j = 0; j < 4; j++) {
    printf("%.1f", col[j] / 3.0);
}
```

按列统计要「4 个累加器」；用一个 sum 反复清零也行，但要存下结果。

累加器变数组

col[4] 就是四个 sum ， col[j] 专门记第 j 门课。

位置换了

累积动作放在内层，因为同一列要跨行累加。

四门课平均

```
72.0  82.7  76.0  73.7
```

找最大元素和它的位置

打擂台升级版

哈爾濱工程大學

```
// s 里已存好 3 行 4 列的成绩
int i, j;
int max = s[0][0];      // 第一个上台
int row = 0, col = 0;   // 记住位置
for (i = 0; i < 3; i++) {
    for (j = 0; j < 4; j++) {
        if (s[i][j] > max) {
            max = s[i][j];
            row = i;  col = j;
        }
    }
}
```

三个变量必须一起更新：只改 max 不改 row/col ，位置就留在旧擂主那里。

还是打擂台

第 07 讲那套：来一个比一个，赢了就换人。

三个变量分工

max 存最大值， row 存行， col 存列 —— 一个都不能少。

初值取 s[0][0] ，保证第一个数也会被拿来比。

本例结果

```
max = 91
row = 0,  col = 2
```

矩阵转置：行列互换

b[j][i] = a[i][j]

```
int a[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
int b[3][2];   //  行列表反着开
int i, j;
for (i = 0; i < 2; i++) {
    for (j = 0; j < 3; j++) {
        b[j][i] = a[i][j];
    }
}
```

2 行 3 列的 a，转置后变成 3 行 2 列的 b —— 转置不改个数，只换排法。

谁变谁不变

元素个数不变，只是 i 和 j 的位置对调了一下。

b 的大小要反着开

a 是 [2][3]，b 就必须是 [3][2]，开小了会越界。

转置后的 b

1	4
2	5
3	6

两个矩阵相加

对应位置，一对一加

```
int a[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
int b[2][3] = {{7, 8, 9}, {1, 1, 1}};
int c[2][3], i, j;
for (i = 0; i < 2; i++) {
    for (j = 0; j < 3; j++) {
        c[i][j] = a[i][j] + b[i][j];
    }
}
```

三个数组必须同型：都是 [2][3] 。行数、列数对不上的两个表没法一对一加。

下标要一致

c[i][j] 只和 a[i][j] 、 b[i][j] 相加，不串门。

一次算一格

内层只有一句：把两个对应位置的数加起来，存进 c 。

相加后的 c

```
 8  10  12
 5   6   7
```

03

字符数组

字符数组 · 字符串 · 结束标志 · 读写 · 多个名字

字符排成队就是字符串，队尾必须有个句号

哈爾濱工程大學

字符数组的定义与初始化

一格一个字符

哈爾濱工程大學

```
// 开 6 个格子，一格放一个字符
char s[6];
s[0] = 'h';
s[1] = 'e';
s[2] = 'l';
s[3] = 'l';
s[4] = 'o';
s[5] = '\0';   // 结束标志
// 定义时直接初始化也行
char t[6] = {'h', 'e', 'l', 'l',
             'o', '\0'};
```

字符常量用单引号，一个单引号只包一个字符； char 的每一格刚好存一个。

单引号 vs 双引号

单引号里是「一个字符」，双引号里是「一串字符」。

下标照样从 0 起

s[0] 是第一个字符，和一维数组一模一样。

多开的那一格

存 5 个字符，为什么要开 6 格？答案在下一页。

末尾的结束标志 '\0'

本课最重要的一个字符

哈尔滨工程大学

字符数组 + 末尾的结束标志 = 字符串。它告诉程序：读到这里就停。

c	h	i	n	a	'\0'
s[0]	s[1]	s[2]	s[3]	s[4]	s[5] 停

printf("%s", s) 从 s[0] 开始一个字符一个字符往外送，遇到结束标志就不送了。

✓ 有它

输出 china ，干净利落，多一格也不用管。

✗ 没它

程序不知道停在哪，接着往下读，后面全是乱码。

它看不见，屏幕上什么都不显示，但它是真的占了一格：存 5 个字符，至少开 6 格。

口诀：要存 n 个字符，就开 n + 1 格。

用 scanf("%s") 读一个单词

不用写 &

哈尔滨工程大学

```c
char name[20];        // 20 格
scanf("%s", name);    // 注意没有 &
printf("%s", name);   // 原样输出
// 为什么不加 &：数组名就是地址
// 第 12 讲指针会讲透，先照写
// 如果输入：hello world
// name 里只有 hello —— 遇空格就停
// world 还留在缓冲区，下次再读
// 想读一整行（带空格）另有办法
// 20 格最多存 19 个字符
// 超了就踩到别人的格子
```

好消息：scanf 会自动在末尾补上结束标志，不用你自己写。

为什么不用 &

数组名本身就代表这一排格子的起点，scanf 知道往哪放。

遇空格就停

%s 一次只读一个「单词」，读不了带空格的整句。

空间宁大勿小

开 20 格就别输入超过 19 个字母，否则越界。

逐字符输出，读到就停

还是第 08 讲的遍历

哈尔滨工程大学

```
char s[10] = "hello";
int i, n = 0;
for (i = 0; s[i] != '\0'; i++) {
    printf("%c", s[i]);  //  一个一个打
    n++;                 //  顺便数个数
}
//  输出 hello，n 等于 5
//  循环条件不是 i < 10
//  而是「还没读到结束标志」
//  这样数组开多大都不影响结果
//  也不用事先知道有几个字符
```

用 while 也完全可以：while (s[i] != 结束标志）一样能走完整个字符串。

%c 输出一个字符

%c 一次一个，%s 一次一串 —— 别混着用。

循环到哪停

条件是 s[i] 不等于结束标志，不是 i 小于 10。

顺手能做什么

数个数、判大小写、找某个字母 —— 都在这个循环里。

双引号初始化与内存示意

```
char s[] = "hello";     //  省略长度
//  编译器数出来要  6  格:  5  +  结束标志
char t[6] = "hello";    //  自己写  6
char u[10] = "hello";   //  后面  4  格空着
//  双引号初始化:结束标志自动补
//  单引号一个个写:要自己补
char v[6] = {'h', 'e', 'l', 'l',
             'o', '\0'};
char w[5] = "hello";    //  少开一格
//  放不下结束标志,输出全是乱码
```

推荐写成 char s[20] = "hello" ：位置留足，又不必自己数。

s[] = "hello" 的 6 格

h	e	l	l	o	'\0'

s[0] ~ s[5] ，最后一格是结束标志。

省略长度

方括号留空，编译器按内容自动开够 6 格。

格子要给够

char w[5] 装 hello 就没地方放结束标志了。

二维字符数组： char name[3][20]

```c
char name[3][20];  // 3 行，每行 20 格
int i;
for (i = 0; i < 3; i++) {
    scanf("%s", name[i]);   // 读一个名字
}
for (i = 0; i < 3; i++) {
    printf("%s", name[i]);  // 输出一个
}
// name[0] 是整行，不是一个字符
// name[0][0] 才是第一个字母
// 每行 20 格：19 个字符 + 结束标志
```

一行就是一个字符串： name[i] 交给 %s ，正好输出第 i 个名字。

一行一个名字

第一维是「第几个名字」，第二维是「这一行的第几格」。

20 是每行的容量

名字最长 19 个字母，超过就把下一行踩了。

输出效果

```
zhangsan
lisi
wangwu
```

例 1 · 成绩表输入与输出

题目 读入 3 个学生 4 门课的成绩，存进二维数组，再按 3 行 4 列打印出来。

```
int s[3][4], i, j;
for (i = 0; i < 3; i++) {
    for (j = 0; j < 4; j++) {
        scanf("%d", &s[i][j]);
    }
}
for (i = 0; i < 3; i++) {
    for (j = 0; j < 4; j++) {
        printf("%4d", s[i][j]);
    }
    //  这一行打完，输出一个换行
}
```

运行结果

输入 12 个整数后输出：

```
  72   85   91   64
  66   90   55   88
  78   73   82   69
```

为什么用 %4d

每个数占 4 格，两位数三位数都能左右对齐。

例 2 · 求每个学生的平均分

题目 在例 1 的成绩表上，算出每个学生 4 门课的平均分，一人一行。

```
// s[3][4] 里已经存好成绩
int i, j;
float sum;
for (i = 0; i < 3; i++) {
    sum = 0;                    // 换人，清零
    for (j = 0; j < 4; j++) {
        sum = sum + s[i][j];
    }
    printf("%.1f", sum / 4.0);
    // 一个人的平均分打完，换行
}
```

运行结果

三个人各占一行：

```
78.0
74.8
75.5
```

除以 4.0 不是 4

写成 4 就是整数除法，小数部分直接被丢掉。

例 3·3×3 矩阵转置

题目 读入一个 3×3 矩阵，把它转置后存入另一个数组并输出。

```
int a[3][3], b[3][3];
int i, j;
// 先读入 a 的 9 个数（双重 for）
for (i = 0; i < 3; i++) {
    for (j = 0; j < 3; j++) {
        b[j][i] = a[i][j];  // 行列对调
    }
}
// 再按行输出 b 的 9 个数
// 输入：1 2 3 / 4 5 6 / 7 8 9
// 输出：1 4 7 / 2 5 8 / 3 6 9
// 对角线上的 1 5 9 位置不变
```

输入 / 输出

输入 a：

```
1 2 3
4 5 6
7 8 9
```

输出 b：

```
1 4 7
2 5 8
3 6 9
```

写 b 的时候，把两个下标换个位置就行。

例 4 · 找最大元素及行列位置

题目 在 3×3 矩阵中找出最大值，同时给出它在第几行、第几列。

```
int a[3][3], i, j, max, row, col;
max = a[0][0];
row = 0;  col = 0;
for (i = 0; i < 3; i++) {
    for (j = 0; j < 3; j++) {
        if (a[i][j] > max) {
            max = a[i][j];
            row = i;  col = j;
        }
    }
}
printf("%d", max);
```

运行结果

输入矩阵：

```
  12  45   7
  33  99  21
   8  56  40
```

输出：

```
max = 99
row = 1, col = 2
```

最大值在第 2 行第 3 列 —— 下标从 0 数，就是 [1][2]。

例 5 · 存 3 个姓名并输出

题目 用二维字符数组存 3 个姓名，每个名字占一行，再一行一个打印。

```
char name[3][20];   // 3 个名字位
int i;
for (i = 0; i < 3; i++) {
    scanf("%s", name[i]);
}
for (i = 0; i < 3; i++) {
    printf("%s", name[i]);
    // 一个名字占一行
}
// 输入: zhangsan lisi wangwu
// 输出三行，每行一个名字
// name[i] 是整行，不是单个字母
```

运行结果

输出：

```
zhangsan
lisi
wangwu
```

三个名字分别躺在 name[0]、name[1]、name[2] 三行里，各带一个结束标志。

常见错误盘点

这 6 条几乎每次作业都有人踩

哈尔滨工程大学

✗ **行列写反** 把 a[i][j] 写成 a[j][i] ，3 行 4 列的表当成 4 行 3 列用。

✗ **越界访问** int a[3][4] 合法下标是 a[0][0] ~ a[2][3] ，写 a[3][0] 就出界了。

✗ **初始化省略列数** int a[][4] 可以，行数让编译器数； int a[3][] 不行，列数不能省。

✗ **双重循环边界写错** 外层写 4 、内层写 3 ，两边对不上，读到别人的格子去了。

✗ **没给结束标志留位置** char s[5] 存 hello 刚好装满，结束标志没地方放。

✗ **没有结尾还用 %s** 数组里没有结束标志却交给 printf("%s") ，它会一直读到乱码。

小结与预告

- 二维数组：两个下标，内存里按行连续存放
- 双重 for ：外层走行、内层走列，累加器位置决定口径
- 字符数组：末尾必须有结束标志， %s 读入输出、 %c 逐个走

下一讲 · 第 10 讲

函数 —— 把一段逻辑起个名字

写一次，用无数次；今天这些「求平均、找最大」终于能收进一个名字里。

哈爾濱工程大學